

1일차: 컨테이너 기술 심화 및 CI/CD 파이프라인

이 장에서는 Dockerfile 최적화와 멀티스테이지 빌드, Docker Compose를 활용한 다중 서비스 관리, GitHub Actions 기반 CI/CD 파이프라인 구성, AWS/GCP 컨테이너 서비스 활용, Blue-Green 배포 전략 등을 자세히 다룹니다. 각 주제별로 이론 설명과 함께 실습 예제, 코드, 결과 예시 및 팁을 제공합니다.

1. Docker 컨테이너 기술 심화

Dockerfile 최적화 및 멀티스테이지 빌드

Docker 이미지는 Dockerfile의 각 명령(`RUN`, `COPY` 등)마다 레이어를 생성합니다. 따라서 이미지 크기를 줄이려면 불필요한 파일이나 빌드 도구를 제거해야 합니다. **멀티스테이지 빌드(multi-stage build)**를 사용하면 빌드 환경과 런타임 환경을 분리하여 최종 이미지에 실행에 필요한 최소 구성만 포함할 수 있습니다 ¹ ². 예를 들어 Go나 Node.js 애플리케이션의 컴파일 결과물만 최종 이미지에 복사하여, 소스 코드나 빌드 도구는 이미지에 포함하지 않을 수 있습니다.

Docker 공식 문서에도 멀티스테이지 빌드는 Dockerfile 최적화를 위해 유용하다고 안내합니다. 예를 들어, Golang 애플리케이션을 빌드한 후 `scratch` 이미지를 사용하여 빌드 결과물만 복사하면, 빌드 도구나 불필요한 파일 없이 매우 경량의 프로덕션 이미지를 만들 수 있습니다 ³ ¹. 다음은 간단한 예시입니다.

```
# --- 빌드용 스테이지 ---
FROM golang:1.24 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp ./main.go

# --- 런타임 스테이지 (최종 이미지) ---
FROM scratch
COPY --from=builder /app/myapp /myapp
CMD ["/myapp"]
```

위 예제에서 첫 번째 `FROM` 명령은 Go SDK가 포함된 빌드용 환경을 만들고, 애플리케이션을 컴파일합니다. 두 번째 `FROM scratch` 단계에서는 이전 스테이지의 컴파일 결과물(`myapp`)만 복사합니다. 결과적으로 Go SDK나 기타 불필요한 파일 없이 바이너리만 담긴 작은 이미지를 얻을 수 있습니다. 멀티스테이지 빌드를 사용하면 이미지 크기를 크게 줄일 수 있고, 보안 관점에서도 빌드 관련 파일 노출을 최소화할 수 있습니다 ¹ ³.

도커이미지 최적화 팁:

- 가능한 한 작은 베이스 이미지를 사용합니다(ex. `alpine` 계열).
- `RUN` 명령을 체인으로 연결(`&&`)하여 불필요한 임시 레이어를 줄입니다.
- 빌드 시 생성되는 캐시 파일이나 테스트용 파일은 `.dockerignore`에 추가해 컨텍스트에 포함되지 않도록 합니다.
- 멀티스테이지 빌드로 빌드 도구를 분리하고, 최종 이미지에는 런타임에만 필요한 아티팩트만 담습니다 ¹ ³.

Docker Compose를 활용한 다중 서비스 관리

Docker Compose는 `docker-compose.yml` 파일 하나로 여러 컨테이너(서비스)를 함께 정의하고 실행할 수 있게 해줍니다. 다중 컨테이너 애플리케이션의 모든 서비스(예: 웹, API, DB, 캐시 등)에 대한 설정을 한곳에 모아 문서화하며, 한 번의 명령어로 전체 애플리케이션을 띄울 수 있습니다 ⁴. Microsoft 문서에 따르면 `docker-compose.yml` 파일은 애플리케이션의 모든 서비스 종속성을 구성·문서화할 수 있는 편리한 도구입니다 ⁴.

예를 들어, 간단한 웹 서비스(Flask)와 MySQL DB를 다루는 `docker-compose.yml` 예시는 다음과 같습니다.

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - db
    environment:
      - FLASK_ENV=production
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: example
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

위 설정에서 `web` 서비스는 현재 디렉토리의 Dockerfile로부터 이미지를 빌드하여 8000번 포트를 노출하고, `db` 서비스가 준비될 때까지 기다립니다. `db` 서비스는 MySQL 이미지를 사용하며 데이터 볼륨 `db_data`를 마운트합니다. 이처럼 Compose를 사용하면 복잡한 연관 서비스를 쉽게 정의하고 `docker-compose up -d` 한 줄로 전체 스택을 시작할 수 있습니다 ⁴.

Compose 활용 팁:

- `depends_on`으로 서비스 시작 순서를 지정할 수 있습니다.
- 환경별로 `docker-compose.override.yml` 파일을 만들어 개발/테스트/프로덕션 설정을 분리할 수 있습니다.
- 테스트용 엔드포인트를 구성하거나 데이터베이스를 떼어놓고 재사용하려면 여러 Compose 파일을 조합할 수 있습니다 ⁵.
- 변경된 서비스만 재시작하려면 `docker-compose up -d 서비스명` 명령을 사용합니다.

실습: 최적화된 Dockerfile 작성 및 실행

1. **프로젝트 준비:** 간단한 Python 웹 애플리케이션(예: Flask)을 준비하고 `app/` 폴더에 소스 코드를 둡니다.
2. **Dockerfile 작성:** 다음과 같이 멀티스테이지 빌드를 사용한 Dockerfile을 작성합니다.

```
FROM python:3.9-slim AS builder
WORKDIR /app
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt --target /app

FROM python:3.9-slim
WORKDIR /app
COPY --from=builder /app /app
COPY app.py .
CMD ["python", "app.py"]
```

3. 첫 번째 스테이지에서 `requirements.txt` 를 기반으로 라이브러리를 `/app` 에 설치합니다.
4. 두 번째 스테이지에서는 빌드 스테이지 결과(`/app` 라이브러리)와 애플리케이션 코드만 복사하여 런타임 이미지를 만듭니다.
5. **빌드 및 실행:** 터미널에서 해당 Dockerfile이 있는 디렉토리로 이동하여 다음 명령을 실행합니다.

```
docker build -t my-flask-app .
docker run -d -p 8000:5000 --name myapp my-flask-app
```

6. `docker build` 로 이미지를 빌드하면 멀티스테이지로 인한 빌드 단계가 출력됩니다.
7. `docker run` 후 `docker ps` 를 통해 컨테이너가 정상 실행 중인지 확인합니다. 웹 브라우저에서 `http://localhost:8000` 을 열어 애플리케이션이 동작하는지 확인합니다.
8. **결과 확인:** 웹 애플리케이션이 정상적으로 응답하면 멀티스테이지 Dockerfile이 올바르게 작성된 것입니다. 최종 이미지 크기를 `docker images` 로 확인하면 빌드 도구가 제외되어 상대적으로 작음을 확인할 수 있습니다.
9. **문제 해결 팁:**
10. **라이브러리 누락 오류:** 종속성 설치가 빠져 있으면 실행 시 모듈 누락 에러가 발생합니다. `pip install` 시 `--target` 혹은 `-t` 옵션을 적절히 사용하거나 두 스테이지의 `WORKDIR` 이 다를 수 있으므로 경로를 확인하세요.
11. **경로 문제:** 두 번째 스테이지에서 `COPY` 경로를 잘못 지정하면 파일 복사 오류가 납니다. 스테이지 이름 (`builder`)을 지정하고 `COPY --from=builder` 로 참조하세요.
12. **.dockerignore 설정:** 소스 코드에 크거나 불필요한 파일(예: `.git`, `node_modules`)이 포함되어 있으면 빌드 컨텍스트가 커져 빌드 속도가 느려집니다. `.dockerignore` 파일을 활용해 필수 파일만 복사되도록 합니다.

2. GitHub Actions 기반 CI/CD 파이프라인

고급 Workflow 구조 이해 및 작성

GitHub Actions는 소스코드 저장소 이벤트(예: `push`, `pull_request`)에 반응하여 테스트, 빌드, 배포 작업을 자동으로 실행하는 CI/CD 플랫폼입니다⁶. 워크플로우는 리포지토리 내 `.github/workflows/` 디렉터리에 YAML 파일로 정의합니다. 각 워크플로우는 하나 이상의 **작업(job)**으로 구성되며, 각 작업은 하나 이상의 **단계(step)**를 실행합니다. 기본적으로 작업들은 병렬로 실행되지만, `needs:` 키워드로 의존성을 지정하여 순차 실행되도록 할 수 있습니다⁷⁸.

예를 들어, 소프트웨어를 다양한 운영체제(OS)나 언어 버전에서 동시에 테스트하려면 **매트릭스(matrix) 전략**을 사용합니다. 매트릭스를 사용하면 동일한 작업이 여러 변형(예: `python: 3.8, 3.9`)으로 병렬 실행됩니다 ⁹. 또한 `if:` 조건문을 통해 특정 브랜치나 이벤트일 때만 단계나 작업을 실행하도록 제어할 수 있습니다. 예를 들어 `if: github.ref == 'refs/heads/main'`로 메인 브랜치에서만 배포 작업을 수행하게 할 수 있습니다 ¹⁰.

워크플로우 예시: 아래는 Python 코드 저장소에서 `main` 브랜치에 푸시될 때 매트릭스 테스트와 빌드, 그리고 메인 브랜치에 한해 배포 작업까지 수행하는 간단한 워크플로우 예입니다.

```
on:
  push:
    branches: [main]

jobs:
  test:
    name: Run Tests (${{ matrix.python-version }})
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: [3.8, 3.9, 3.10] # 병렬 테스트 환경
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v4
      - with:
          python-version: ${{ matrix.python-version }}
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run tests
        run: pytest

  build:
    name: Build Docker Image
    needs: test # 테스트 성공 후 실행
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Build image
        run: |
          docker build -t myapp:$GITHUB_SHA .
          # (필요시 이미지를 레지스트리로 push하는 단계 추가)

  deploy:
    name: Deploy to Production
    needs: build
    if: github.ref == 'refs/heads/main' # 메인 브랜치에서만 실행
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
```

```
- name: Deploy script
run: echo "Deploy to production environment"
```

위 워크플로우에서 **test** 작업은 Python 3.8, 3.9, 3.10 환경에서 병렬로 테스트를 수행합니다 ⁹. 모든 테스트가 성공하면 **build** 작업이 실행되며, 도커 이미지를 빌드합니다. 그리고 **deploy** 작업은 메인 브랜치에서만 조건부로 실행됩니다 ¹⁰ ⁸.

자동화된 테스트, 빌드, 배포 파이프라인 구성

고급 워크플로우 기능을 활용하여 전체 CI/CD 파이프라인을 자동화할 수 있습니다. 일반적으로 워크플로우는 다음 단계를 포함합니다:

- **코드 검사/테스트:** `pytest`, `flake8` 등으로 유닛 테스트와 린트 검사를 실행하여 코드 품질을 검증합니다.
- **아티팩트 빌드:** 성공적인 테스트 후에 애플리케이션 패키지나 도커 이미지를 빌드합니다. 예를 들어 Python 애플리케이션은 `python setup.py bdist_wheel` 로 빌드하거나, 도커 이미지를 `docker build` 로 생성합니다.
- **캐시 활용:** `actions/cache` 액션을 사용해 종속성을 캐시하면 빌드 시간을 단축할 수 있습니다. 예를 들어 `~/.cache/pip` 를 캐시하면 매 실행마다 라이브러리 설치 시간을 절약할 수 있습니다 ¹¹.
- **보안 스캔:** SAST나 서드파티 도구를 사용해 보안 취약점을 검출하는 단계를 추가할 수 있습니다.
- **배포:** 빌드된 결과물을 클라우드 환경(AWS, GCP 등)에 배포합니다. 이 단계에서는 클라우드 자격증명(secret)이 필요하므로, GitHub Secrets나 Environments를 사용해 자격증명을 안전하게 관리합니다.

각 단계는 워크플로우 내 여러 **jobs**와 **steps**로 구성됩니다. 예를 들어, **job 간 의존성**(`needs:`)을 설정하여 빌드 작업이 테스트 작업 이후에 실행되도록 할 수 있고 ⁸, **매트릭스**로 여러 파이썬 버전을 동시에 처리할 수 있습니다 ⁹. 조건문(`if:`)을 활용하면 테스트 통과 시에만 다음 작업이 실행되거나 특정 브랜치에서만 배포 작업이 동작하도록 구성할 수 있습니다 ¹⁰.

실습: GitHub Actions로 CI/CD 구현

1. **저장소 준비:** GitHub에 Python 애플리케이션 저장소를 생성합니다. 기존 코드를 푸시하거나 새로운 파이썬 프로젝트를 초기화합니다.
2. **워크플로우 파일 생성:** 저장소 루트에 `.github/workflows/ci.yml` 파일을 생성하고, 위 예시와 같이 워크플로우 정의를 작성합니다.
3. **GitHub Secrets 설정:** 만약 배포나 도커 레지스트리 액션에 클라우드 자격증명이 필요하다면, 저장소 Settings > Secrets에서 `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `GCP_SA_KEY` 등 필요한 시크릿을 등록합니다.
4. **파일 커밋:** 변경 사항을 커밋하고 `main` 브랜치에 푸시합니다. 푸시와 동시에 Actions 탭에서 워크플로우 실행 로그를 확인할 수 있습니다.
5. **결과 확인:** GitHub Actions 페이지에서 각 작업의 실행 상태(성공/실패)를 확인합니다. 빌드나 테스트가 실패하면 로그를 열어 오류 원인을 확인할 수 있습니다.

오류 해결 팁:

- YAML 문법 오류가 있으면 Actions 실행이 시작되지 않거나 워크플로우가 실패합니다. (예: 들여쓰기, 띄어쓰기 실수)
- `uses:` 액션 버전이 올바른지, `with:` 파라미터에 오차는 없는지 확인하세요.
- 비밀키나 환경 변수가 누락되면 인증 오류가 발생합니다. GitHub Secrets에 등록했는지, 이름을 정확히 입력했는지 확인합니다.
- 워크플로우 내 캐시 오류 시 캐시 키 구성(`key:`)이나 경로(`path:`)를 점검하세요.

3. 클라우드 컨테이너 서비스 활용

AWS ECS(Fargate) 및 GCP Cloud Run/GKE 소개

AWS ECS (Elastic Container Service)는 AWS의 컨테이너 오케스트레이션 서비스로, Docker 컨테이너를 자동으로 실행·확장·배포할 수 있도록 지원합니다 ¹². ECS는 컨테이너를 논리적인 그룹인 클러스터에서 운영하며, 컨테이너 설정을 정의한 태스크 정의(Task Definition)를 기반으로 태스크(Task)가 생성됩니다. 또한 ECS 서비스(Service)를 통해 태스크를 자동으로 유지하거나 로드밸런싱으로 트래픽을 분산할 수 있습니다 ¹². ECS는 두 가지 런타임 옵션을 제공하는데, EC2 인스턴스 기반과 **Fargate** 기반입니다. **AWS Fargate**는 서버를 직접 관리할 필요 없는 서버리스 컨테이너 실행 환경입니다 ¹³. Fargate를 사용하면 EC2 인스턴스를 띄우고 관리하는 과정을 AWS가 대행하므로, 사용자 입장에서는 컨테이너 실행에만 집중할 수 있습니다 ¹⁴.

Google Cloud Run은 Google Cloud의 서버리스 컨테이너 플랫폼입니다. Cloud Run을 이용하면 개발자는 컨테이너 이미지를 Google의 확장 가능한 인프라에서 바로 실행할 수 있습니다. Cloud Run은 관리형(fully managed) 서비스로, 인프라 구성이나 클러스터 운영 없이도 컨테이너 서비스를 배포할 수 있습니다 ¹⁵. 예를 들어, Python이나 Go 등 어떤 언어로 작성된 코드를 컨테이너화하여 Cloud Run에 배포하면, 요청에 따라 자동 확장되며 사용한 리소스에 따라 과금됩니다 ¹⁵ ¹⁶.

Google Kubernetes Engine (GKE)는 Kubernetes 기반의 컨테이너 오케스트레이션 서비스를 제공합니다. Kubernetes는 현재 가장 대중적인 오케스트레이션 플랫폼으로, AWS, GCP, Azure 등 주요 클라우드에서 관리형 서비스를 지원합니다 ¹⁷. Kubernetes를 이용하면 여러 머신에 걸친 컨테이너 클러스터를 구성하여 애플리케이션을 배포하고 자동화된 스케일링, 롤링 업데이트, 서비스 디스커버리 등을 구현할 수 있습니다.

요약하면, AWS ECS는 AWS 생태계에서 컨테이너를 관리하는 서비스이고, Fargate는 여기에 서버리스 런타임을 결합한 형태입니다 ¹² ¹⁴. GCP에서는 Cloud Run이 간단한 서버리스 컨테이너 실행 옵션이고, GKE가 Kubernetes 기반의 풀 매니지드 오케스트레이션입니다. 멀티클라우드 환경에서는 각 플랫폼의 장단점을 이해하여 필요한 곳에 적합한 서비스를 선택할 수 있습니다.

컨테이너 오케스트레이션 개념 이해

컨테이너 오케스트레이션은 컨테이너화된 애플리케이션을 **프로비저닝(provisioning)**, **배포**, **확장**, **관리**하는 작업을 자동화하는 기술입니다. 수십 개, 수백 개의 컨테이너를 운영 환경에 맞게 배치하고 장애 대비를 설정하는 등의 반복 작업을 수동으로 하기 어렵기 때문에, 오케스트레이션 도구가 이를 대신 수행합니다 ¹⁸. IBM에 따르면, 컨테이너 오케스트레이션은 컨테이너 애플리케이션의 전 생애주기를 자동으로 관리하여 CI/CD 파이프라인의 핵심 역할을 하며, 최신 하이브리드 멀티클라우드 인프라에서 민첩성과 속도를 제공합니다 ¹⁸.

대표적인 오케스트레이션 플랫폼인 **Kubernetes**는 클러스터 단위로 컨테이너를 관리합니다. 사용자는 Kubernetes 매니페스트(YAML)를 통해 원하는 상태(배포할 컨테이너 이미지, 수, 리소스 요구사항 등)를 선언하면, Kubernetes가 자동으로 노드에 컨테이너를 스케줄링하고 관리합니다 ¹⁸ ¹⁹. 오케스트레이션의 주요 기능에는 자동 **스케일링**, **로드 밸런싱**, **헬스체크** 및 **자기치유(self-healing)**, **롤링 업데이트/롤백** 등이 있습니다. 즉, 인프라 환경을 직접 손대지 않고도 컨테이너를 효율적으로 운영할 수 있게 해줍니다. 실제로 많은 기업에서 Kubernetes와 같은 관리형 오케스트레이션 서비스를 사용하여 서비스를 운영하고 있습니다 ¹⁸ ¹⁷.

실습: 서버리스 컨테이너 배포 (ECS 또는 Cloud Run)

이 실습에서는 이미지를 클라우드로 배포하는 예시를 살펴봅니다. 두 가지 방법 중 하나를 선택해 진행할 수 있습니다.

- **AWS ECS(Fargate) 예시:**
- Docker 이미지를 AWS ECR에 업로드합니다.

```
aws ecr create-repository --repository-name my-app
aws ecr get-login-password --region us-east-1 | docker login --username AWS --
password-stdin 123456789012.dkr.ecr.us-east-1.amazonaws.com
docker build -t my-app .
docker tag my-app:latest 123456789012.dkr.ecr.us-east-1.amazonaws.com/my-
app:latest
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/my-app:latest
```

- AWS ECS 콘솔에서 **클러스터**를 만들고, Fargate를 런타임으로 선택합니다.
- **태스크 정의(Task Definition)**를 생성하여 1단계에서 푸시한 ECR 이미지를 지정합니다.
- **서비스(Service)**를 생성하고, 태스크 정의를 지정한 뒤 원하는 개수만큼 태스크를 실행합니다. 필요시 ALB를 연결하여 로드밸런싱할 수 있습니다.
- 서비스가 기동되고 태스크가 정상 실행되면, 퍼블릭 IP(또는 ALB DNS)를 통해 애플리케이션에 접속합니다.

Fargate는 서버 인프라를 신경쓰지 않아도 되는 서버리스 환경이므로 편리합니다. **팁:** IAM 역할이 적절히 설정되어 있지 않으면 태스크 실행 실패, 로그 전송 실패 등의 에러가 발생할 수 있습니다. 필요한 권한(AmazonECSTaskExecutionRole 등)이 태스크 역할에 부여되었는지 확인하세요.

- **GCP Cloud Run 예시:**
- Google Cloud SDK(`gcloud`)를 설치하고 로그인한 뒤 프로젝트를 설정합니다.
- 코드를 컨테이너로 빌드하고 Container Registry에 푸시합니다. 예:

```
gcloud builds submit --tag gcr.io/PROJECT_ID/my-python-app .
```

- Cloud Run 서비스를 생성하여 방금 빌드한 이미지를 배포합니다. 예:

```
gcloud run deploy my-python-app \
  --image gcr.io/PROJECT_ID/my-python-app \
  --platform managed \
  --region asia-northeast3 \
  --allow-unauthenticated
```

- 배포가 완료되면 Cloud Run 서비스의 URL이 출력됩니다. 브라우저에서 해당 URL에 접속하여 애플리케이션이 정상 동작하는지 확인합니다.

Cloud Run은 코드 작성 이외에 클러스터 관리나 서버 설정을 거의 하지 않아도 됩니다 ¹⁵ ¹⁶. **팁:** Cloud Run 배포 전에 Cloud Run API가 활성화되어 있는지, 서비스 계정 권한이 충분한지 확인하세요. 또한 요금제가 **요청 기반(Request-based)**인지, **인스턴스 기반**인지에 따라 과금 방식이 다르니 운영 환경에 맞게 선택합니다.

4. 자동화된 배포 전략

GitHub Actions와 클라우드 연동

CI/CD 파이프라인에서 GitHub Actions와 클라우드 서비스를 연동하려면 액세스 키나 서비스 계정 키 같은 인증 정보를 사용해야 합니다. 예를 들어 AWS에 배포할 때는 `aws-actions/configure-aws-credentials` 액션을 사용하여 AWS 자격증명을 설정할 수 있습니다. GCP의 경우 `google-github-actions/auth`와 `google-github-actions/setup-gcloud` 등을 사용하여 서비스 계정 키를 이용해 인증합니다. 이러한 자격증명은 GitHub Secrets에 등록해 보관하며 워크플로우에서 참조합니다.

또한, 배포 시 **브랜치나 이벤트에 따라 환경을 분리**하는 것이 중요합니다. 예를 들어 `main` 브랜치로 병합될 때 프로덕션에 배포하고, `dev` 브랜치에서는 스테이징 환경에 배포하도록 워크플로우를 구성할 수 있습니다. GitHub의 **Environments** 기능을 사용하면 `staging`, `production` 등의 이름을 가진 환경을 만들고, 각각의 환경에 대해 별도의 시크릿과 승인을 설정할 수 있습니다. 이로써 중요한 프로덕션 배포는 추가 승인 단계를 거치도록 하여 안정성을 높일 수 있습니다.

환경 분리 전략 (Staging, Production)

스테이징/프로덕션 같은 여러 환경을 분리하려면, 다음과 같은 방식을 활용합니다:

- **브랜치 전략:** 코드 레포에서 `dev`, `staging`, `main` 등으로 브랜치를 분리하고, 각 브랜치 푸시 시 서로 다른 워크플로우가 동작하도록 설정합니다. 예를 들어 `on: push: branches: [dev]` 와 `branches: [main]` 으로 워크플로우 파일을 분리할 수 있습니다.
- **GitHub Environments:** Settings > Environments에서 환경을 생성한 후, 워크플로우의 `environment:` 키를 지정하여 사용합니다. 환경별 시크릿을 달리 설정하거나, 배포 승인(review) 규칙을 걸 수 있습니다.
- **조건부 배포:** `if: github.ref == 'refs/heads/main'` 같은 조건문으로 프로덕션 브랜치에서만 배포 단계가 실행되도록 합니다. 스테이징에서는 다른 조건을 사용할 수 있습니다.

이렇게 설정하면 개발/테스트 단계에서는 자유롭게 배포해보면서도, 최종 프로덕션은 별도 승인이나 엄격한 조건을 거쳐 배포할 수 있습니다.

실습: Blue-Green 배포 및 롤백 설정

Blue-Green 배포는 현재 서비스 중인 환경(Blue)과 새로운 버전을 준비할 환경(Green)을 동시에 운영하다가, 준비가 완료되면 트래픽을 한꺼번에 전환하는 방식입니다 ²⁰. 기존 버전에서 새로운 버전으로 즉시 스위칭하기 때문에 중간 단계 호환성 문제를 줄일 수 있고, 문제가 발생하면 즉시 Blue 환경으로 트래픽을 되돌려 **롤백**할 수 있다는 장점이 있습니다 ²⁰.

예를 들어, AWS 환경에서는 다음과 같이 구성할 수 있습니다:

1. 기존 프로덕션 환경은 **Blue**로 두고, 새로운 버전의 태스크를 포함하는 **Green** 서비스를 별도로 생성합니다. (Green은 Blue와 동일한 설정의 복제본)
2. 두 서비스에 ALB(또는 NLB) 로드밸런서를 연결하여 동일한 도메인/엔드포인트로 라우팅합니다.
3. Green 배포가 완료되면 ALB의 대상 그룹(Target Group) 설정을 수정하거나 서비스 디스커버리를 업데이트하여 트래픽을 Blue에서 Green으로 전환합니다.
4. 문제가 없으면 Blue 환경을 종료합니다. 문제가 생기면 다시 Blue로 스위치하여 롤백합니다.

GCP Cloud Run에서도 비슷한 방식이 가능합니다. Cloud Run은 서비스 내 버전별 트래픽 분할 기능을 제공합니다. 예를 들어 새 버전을 배포할 때 100%를 Green으로 설정하여 전환하거나, 일정 비율을 테스트 후 올리는 방식으로 돌아올 수 있습니다. 문제가 생길 경우 이전 버전으로 쉽게 복귀할 수 있습니다.

실습 예시: (Blue-Green 배포 설정 개략)

```
# AWS CLI 예시: ALB 대상 그룹에서 트래픽 스위칭 (예상 커맨드)
aws elbv2 modify-listener --listener-arn arn:aws:elasticloadbalancing:us-east-1:123456789012:listener/app/alb-listener/12345678901234567890123456789012/12345678901234567890123456789012 --default-actions Type=forward,TargetGroupArn=arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/green-tg/12345678901234567890123456789012

# Cloud Run 예시: 트래픽 100% Green으로 전환
```



```
gcloud run services update-traffic my-python-app \
--to-latest
```

오류 해결 팁: Blue-Green 배포 시에는 두 환경을 동시에 운영하므로 추가 비용이 발생할 수 있습니다. 또한, DNS 캐싱이나 로드밸런서 헬스체크 설정이 올바르지 않으면 트래픽 전환이 기대대로 이루어지지 않습니다. 스위치 전후에 헬스체크 통과 여부와 트래픽 흐름을 반드시 확인해야 합니다.

정리

이상의 내용으로 1일차에는 **Docker 컨테이너 기술**과 **CI/CD 파이프라인**에 관한 심화 내용을 다룹니다. 각 개념을 설명하고, 구체적인 코드 예시와 실습을 통해 이해를 돕습니다. 특히 멀티스테이지 Dockerfile 작성, Docker Compose를 통한 멀티 컨테이너 관리, GitHub Actions 워크플로우 작성 및 실행, AWS/GCP 컨테이너 서비스로의 배포, Blue-Green 무중단 배포 등은 실무에서도 자주 사용되는 기술입니다. 여기에 소개된 예제와 절차를 따라 해보면서 개념을 확실히 익히고, 혹시 발생하는 에러는 로그를 면밀히 분석해 해결해 보시기 바랍니다.

참고 자료: 각 기술의 개념 및 사용법은 공식 문서와 기술 블로그 등을 참고하였습니다 1 4 9 6 12 18 20 .

1 Dockerfile 최적화 Multi-Stage Build(멀티스테이지 빌드)

<https://rae0.tistory.com/49>

2 3 Multi-stage | Docker Docs

<https://docs.docker.com/build/building/multi-stage/>

4 5 docker-compose.yml 사용하여 다중 컨테이너 애플리케이션 정의 - .NET | Microsoft Learn

<https://learn.microsoft.com/ko-kr/dotnet/architecture/microservices/multi-container-microservice-net-applications/multi-container-applications-docker-compose>

6 7 GitHub Actions 이해 - GitHub Docs

<https://docs.github.com/ko/actions/get-started/understand-github-actions>

8 9 10 11 GitHub Actions 고급 워크플로우 구성법

<https://hobbylife.tistory.com/entry/GitHub-Actions-%EA%B3%A0%EA%B8%89-%EC%9B%8C%ED%81%AC%ED%94%8C%EB%A1%9C%EC%9A%B0-%EA%B5%AC%EC%84%B1%EB%B2%95>

12 13 14 [CLOUD] AWS ECS + Fargate 이해

<https://yerim110324.tistory.com/45>

15 16 Cloud Run이란 무엇인가요? | Cloud Run Documentation | Google Cloud

<https://cloud.google.com/run/docs/overview/what-is-cloud-run?hl=ko>

17 18 19 컨테이너 오케스트레이션이란 무엇인가요? | IBM

<https://www.ibm.com/kr-ko/think/topics/container-orchestration>

20 Blue Green 무중단 배포

https://velog.io/@hooni_/Blue-Green-%EB%AC%B4%EC%A4%91%EB%8B%A8-%EB%B0%B0%ED%8F%AC